

MonocularVO

Name: Aarti Krishan Khatri

R Number: 11860380

Institution: Texas Tech University

Date: 12/07/2025

Abstract

This report presents *MonocularVO*, a modular monocular visual odometry (VO) system designed for real-world driving scenarios and evaluated on a KITTI Raw sequence. The pipeline integrates ORB feature extraction, Essential-matrix-based pose initialization, and a lightweight PnP reprojection-error refinement to obtain geometrically consistent frame-to-frame motion estimates in $SE(3)$. Global trajectory reconstruction is further improved through a Kalman-based temporal smoothing module that reduces translation noise while preserving overall motion structure. The system is assessed using qualitative trajectory visualizations and internal consistency statistics. Ablation and sensitivity studies highlight the influence of smoothing, RANSAC thresholds, and feature count on path stability and drift. The results demonstrate that classical geometric estimation combined with lightweight optimization yields a coherent and robust VO trajectory.

1 Introduction

Visual odometry (VO) estimates a camera’s ego-motion using only visual input and is a core component of autonomous driving, robotics, AR/VR, and mobile perception systems. Unlike full SLAM pipelines, which maintain a global map of the environment, VO prioritizes efficient frame-to-frame motion estimation and trajectory reconstruction. This project implements a complete, modular monocular VO pipeline called MonocularVO and evaluates it on the KITTI Raw dataset.

The goal of the system is to estimate relative motion between consecutive frames, accumulate poses into a consistent global trajectory in $SE(3)$, and apply a principled optimization method to reduce measurement noise and improve temporal consistency.

1.1 Dataset Choice

The system is evaluated on the KITTI Raw City sequence `2011_09_26_drive_0005_sync`, a real-world forward-driving sequence captured from a vehicle moving through an urban environment. This sequence contains realistic motion patterns, dynamic vehicles, illumination variation, and moderate camera rotation, making it a suitable and challenging benchmark for feature-based VO. Camera intrinsics are extracted from the KITTI calibration file

`calib_cam_to_cam.txt`, explicitly using the rectified left camera projection matrix P_{00} , which yields the intrinsic matrix K required for epipolar geometry and PnP refinement.

1.2 Problem Definition

Given a sequence of monocular images

$$\{I_0, I_1, \dots, I_N\}$$

and camera intrinsics K , the objective is to estimate a sequence of camera poses

$$T_k = \begin{bmatrix} R_k & t_k \\ 0 & 1 \end{bmatrix} \in SE(3).$$

The system must:

- Detect and match keypoints between consecutive frames,
- Estimate relative pose using geometric constraints,
- Accumulate poses into a global trajectory,
- Apply a non-trivial optimization or smoothing method,
- Visualize the final 3D trajectory.

Because this VO system is monocular, the recovered trajectory is up to scale, but relative motion and structure must remain consistent.

1.3 Key Idea and Novelty

The proposed system, *MonocularVO*, follows a classical feature-based monocular VO pipeline while adding a lightweight geometric refinement and temporal smoothing stage. Instead of pure essential-matrix pose recovery, the method explicitly optimizes reprojection error and then stabilizes the translation estimates with a Kalman-based smoother.

Concretely, each frame-to-frame motion estimate is produced in two stages:

1. **Geometric pose initialization via Essential matrix.** ORB features are detected and matched between consecutive frames. Using these correspondences, an Essential matrix is estimated with RANSAC to reject outliers, and an initial relative pose (R, t) is recovered in $SE(3)$. This provides a robust but noisy estimate of the camera motion.
2. **Reprojection-error minimization via PnP refinement.** Using inlier correspondences, 3D landmarks are triangulated and a PnP optimization problem is solved:

$$\min_{R, t} \sum_i \|\mathbf{u}_i - \pi(R\mathbf{X}_i + t)\|^2,$$

where $\pi(\cdot)$ is the camera projection function, $\mathbf{X}_i$ are 3D points in the world frame, and $\mathbf{u}_i$ are their observed image measurements. This refinement step reduces reprojection error and moves the system closer in spirit to backend SLAM techniques such as local bundle adjustment, while remaining computationally lightweight.

After geometric refinement, the per-frame translation estimates are passed through a 3D Kalman smoother that models camera motion as a noisy dynamical system. This temporal optimization reduces frame-to-frame jitter without simply averaging positions and produces a more consistent global trajectory in $SE(3)$ suitable for downstream evaluation and visualization.

1.4 Background Context

Feature-based VO has been extensively studied through classical formulations such as the eight-point algorithm, essential matrix decomposition, RANSAC outlier rejection, triangulation, and PnP solvers. Modern SLAM systems, including ORB-SLAM [1] and VISO2/StereoScan [2], integrate these

geometric components with backend optimization, such as bundle adjustment, to minimize reprojection error and produce globally consistent trajectories.

This project adopts several of these principles:

- Keypoint detection and matching form the *front-end*,
- Essential matrix estimation and PnP refinement form the *geometric back-end*,
- A Kalman filter provides *temporal optimization* over the trajectory.

This modular design aligns with practical VO/SLAM architectures and demonstrates how optimization can improve temporal consistency without implementing a full bundle-adjustment SLAM pipeline.

2 Design Overview

This section presents the high-level architecture of the proposed MonocularVO system. The design follows a modular structure inspired by modern SLAM pipelines, separating the front-end (feature extraction and geometric estimation) from the back-end (optimization, filtering, and trajectory computation). Figure 1 illustrates the full data flow.

2.1 Pipeline Architecture

The system follows a sequential front-end/back-end VO pipeline:

1. **Image Loading** The frames and calibration parameters are loaded from the KITTI Raw sequence.
2. **Feature Extraction & Matching** ORB keypoints and descriptors are extracted and matched using a brute-force Hamming matcher.
3. **Relative Pose Estimation** The Essential matrix is computed via RANSAC, and its decomposition yields an initial estimate of rotation R and translation t .
4. **Pose Refinement (Optimization)** Triangulated 3D points and 2D observations are passed to an iterative PnP solver to minimize the reprojection error.
5. **Trajectory Accumulation** The refined poses are composed into a global $SE(3)$ trajectory.

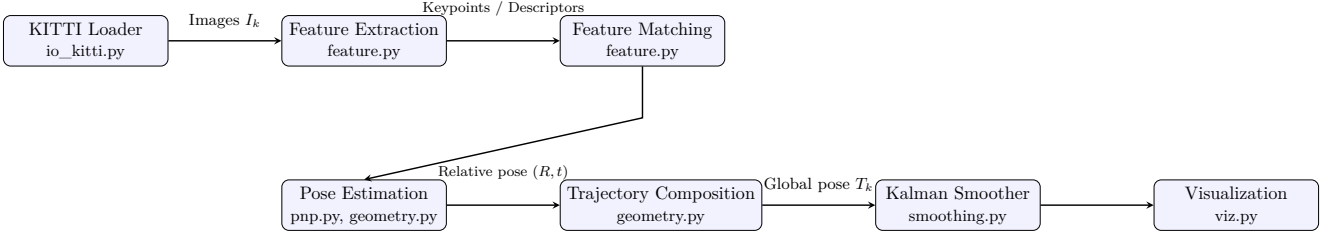


Figure 1: High-level system architecture showing the modules and data flow in the proposed VO pipeline.

6. **Temporal Smoothing / Filtering** A Kalman filter regularizes noisy translation estimates.

7. **Visualization** 2D and 3D trajectories, orientations, and frustum plots are generated.

2.2 Modules and Data Flow

Each component in the pipeline is implemented as an independent module in `src/vo/`, keeping the codebase modular and making it easy to test or replace individual stages without changing the rest of the system.

- **io_kitti.py** Loads KITTI images, timestamps, and camera intrinsics from `calib_cam_to_cam.txt`. It provides a simple iterator-style interface that returns grayscale frames and the intrinsic matrix K , which serves as the input to the rest of the pipeline.
- **feature.py** Performs ORB keypoint extraction + descriptor computation, and uses a BFMatcher with Hamming distance for feature matching. It exposes high-level functions that return keypoints, descriptors, and filtered match lists that the geometry and pose-estimation modules can directly consume.
- **geometry.py** Implements SE(3) utilities such as homogeneous transforms, pose composition, and coordinate-frame conversions. This module centralizes all rigid-body transform math so that the rest of the code works with clean 4×4 poses rather than ad hoc matrix operations.
- **pnp.py** Handles Essential matrix estimation with RANSAC and subsequent pose recovery, followed by PnP-based reprojection-error refinement. It serves as the main geometric backend: given matched features and triangulated 3D points, it computes a refined relative pose (R, t) using inlier masks from the Essential matrix estimation.

- **smoothing.py** Implements a 3D Kalman filter that enforces temporal consistency over the camera translation states. This module treats raw VO translations as noisy measurements and produces a filtered trajectory for visualization and analysis.
- **optimizer.py** Contains utility functions for reporting trajectory statistics such as path length, per-step motion, and relative differences between raw and filtered trajectories. These summaries are used in the evaluation section to reason about the effect of the optimization/smoothing stage.
- **viz.py** Provides all plotting utilities, including a 2D top-down X-Z plot, a full 3D trajectory view, and frustum/orientation visualizations along the path.

Data flows linearly through these modules, with the PnP refinement and Kalman smoothing forming the core optimization pipeline that improves the accuracy and stability of the estimated trajectory.

2.3 Where Optimization Fits

Optimization occurs at two levels:

1. Geometric Optimization (Frame-Level)

The PnP module refines the raw (R, t) estimates by minimizing reprojection error:

$$\min_{R, t} \sum_i \|u_i - \pi(RX_i + t)\|^2$$

improving geometric consistency.

2. Temporal Optimization (Trajectory-Level)

The Kalman filter models the camera motion as:

$$x_k = Ax_{k-1} + w_k, \quad z_k = Hx_k + v_k$$

smoothing frame-to-frame noise while preserving motion structure.

These two optimization layers together produce a cleaner, more stable trajectory without requiring full bundle adjustment.

3 Detailed Design

This section describes the mathematical and algorithmic foundations of the visual odometry system, *MonocularVO*. The pipeline consists of feature extraction, epipolar-geometry-based pose initialization, PnP-based reprojection-error refinement, SE(3) trajectory composition, and temporal smoothing via a Kalman filter. Each component is presented with its underlying equations and design choices.

3.1 Camera Model

A pinhole camera model is assumed. A 3D point $X = [X, Y, Z]^T$ in the camera frame is projected onto the image plane using:

$$u = \pi(X) = \begin{bmatrix} f_x \frac{X}{Z} + c_x \\ f_y \frac{Y}{Z} + c_y \end{bmatrix},$$

where K is the KITTI intrinsic calibration matrix:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}.$$

Pixel coordinates are converted to normalized coordinates using:

$$\tilde{x} = K^{-1}u.$$

This model is used throughout the system for epipolar geometry estimation, triangulation, and reprojection-error optimization.

3.2 Feature Extraction and Matching

ORB features are used for keypoint detection and descriptor computation. For each image I_k , the extractor outputs:

$$\{u_i^k\}, \quad \{d_i^k\},$$

where u_i^k are keypoint locations and d_i^k are binary descriptors.

Feature correspondences between consecutive frames (I_{k-1}, I_k) are found using:

$$(d_i^{k-1}, d_j^k) \rightarrow \text{Hamming distance minimization.}$$

Outliers are filtered via ratio test and geometric RANSAC.

3.3 Essential Matrix Estimation and Pose Recovery

Given matched image points u_i, u'_i and camera intrinsics K , the corresponding normalized coordinates $(\tilde{x}_i, \tilde{x}'_i)$ satisfy the epipolar constraint:

$$\tilde{x}'_i{}^T E \tilde{x}_i = 0.$$

In the implementation, E is estimated using OpenCV's `findEssentialMat` with RANSAC and the known intrinsics K , which internally applies a calibrated 5-point method to enforce this constraint and reject outliers:

$$E = \arg \min_E \sum_i \left| \tilde{x}'_i{}^T E \tilde{x}_i \right|.$$

Once E is obtained, the relative pose (R, t) is extracted from

$$E = [t]_{\times} R.$$

Four candidate (R, t) pairs arise; the correct one is determined by enforcing positive depth after triangulation:

$$Z_i > 0 \quad \forall i.$$

This produces an initial estimate of camera motion between frames $k-1$ and k .

3.4 PnP-Based Reprojection-Error Refinement

Given inlier correspondences, 3D points X_i are triangulated using the relative pose between the two frames. The first camera is fixed at $[I \mid 0]$ and the second at $[R \mid t]$:

$$X_i = \text{Triangulate}(u_i^{k-1}, u_i^k, R, t),$$

which in practice corresponds to the projection matrices $P_1 = K[I \mid 0]$ and $P_2 = K[R \mid t]$.

A perspective- n -point (PnP) optimization is then solved to refine the pose:

$$\min_{R, t} \sum_i \left\| u_i^k - \pi(RX_i + t) \right\|^2,$$

where $\pi(\cdot)$ is the camera projection function. OpenCV's iterative PnP algorithm performs a non-linear refinement of this objective, acting as a lightweight reprojection-error optimization step.

This refinement step reduces reprojection error and improves robustness in dynamic or low-texture areas.

3.5 SE(3) Pose Representation and Composition

A camera pose is represented as a homogeneous transformation:

$$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \in SE(3).$$

The global pose is obtained by composing relative motions:

$$T_k = T_{k-1} \cdot \Delta T_k.$$

Although the implementation does not explicitly use Lie algebra $\mathfrak{se}(3)$, the pose operations follow SE(3)'s group structure. The exponential and logarithm maps are not required, but the system respects:

$$SE(3) : (R, t) \circ (R', t') = (RR', Rt' + t).$$

This guarantees consistent global trajectory accumulation.

3.6 Temporal Optimization via Kalman Smoothing

To reduce frame-to-frame jitter, a Kalman filter is applied to the translation component of the trajectory. Let:

$$x_k = t_k, \quad z_k = \hat{t}_k^{VO},$$

where z_k is the noisy VO estimate.

3.6.1 Motion Model

A constant-position model is used:

$$x_k = x_{k-1} + w_k, \quad w_k \sim \mathcal{N}(0, Q).$$

3.6.2 Measurement Model

$$z_k = x_k + v_k, \quad v_k \sim \mathcal{N}(0, R).$$

3.6.3 Prediction Step

$$\hat{x}_{k|k-1} = \hat{x}_{k-1},$$

$$P_{k|k-1} = P_{k-1} + Q.$$

3.6.4 Update Step

Kalman gain:

$$K_k = P_{k|k-1} (P_{k|k-1} + R)^{-1}.$$

State update:

$$\hat{x}_k = \hat{x}_{k|k-1} + K_k (z_k - \hat{x}_{k|k-1}).$$

Covariance update:

$$P_k = (I - K_k) P_{k|k-1}.$$

This smoother provides a principled probabilistic refinement, improving temporal consistency without altering rotation estimates.

3.7 Final Trajectory Construction

The refined translation and estimated rotations are combined to produce the final global poses:

$$T_k^{final} = \begin{bmatrix} R_k & \hat{x}_k \\ 0 & 1 \end{bmatrix}.$$

These poses are plotted in both 2D and 3D using `viz.py`, completing the VO pipeline.

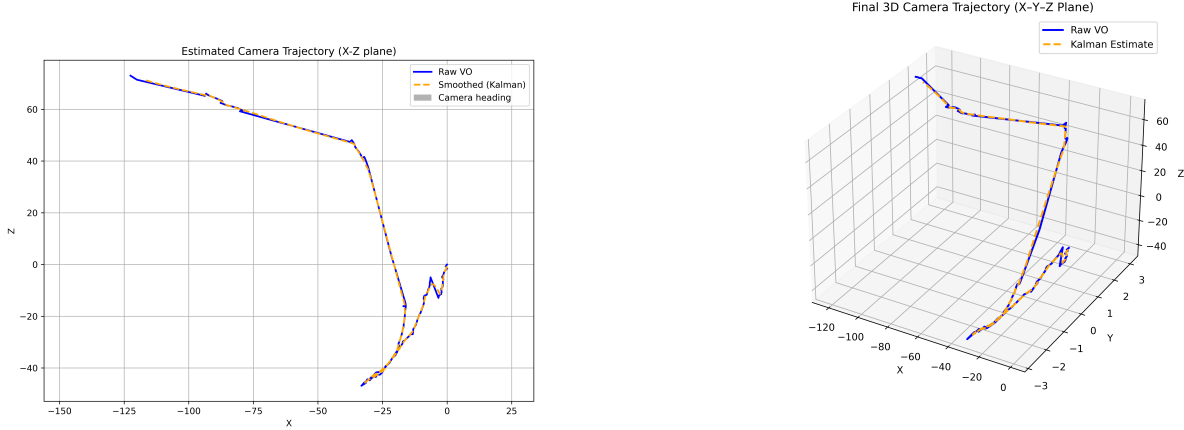
4 Evaluation Results

This section evaluates the behavior of the MonocularVO system on the 2011_09_26_drive_0005_sync sequence. The system focuses on qualitative trajectory analysis and internal consistency statistics rather than quantitative error metrics against ground truth. The results are reported for both the raw visual odometry (VO) estimates and the Kalman-filtered trajectory.

4.1 Trajectory Visualizations

Figure 2 shows the main trajectory visualizations. The top-down X-Z plot (left) overlays the raw VO trajectory with the Kalman-filtered estimate, and includes heading arrows that indicate the estimated camera forward direction. The 3D plot (right) displays the trajectory in X-Y-Z, highlighting that the recovered motion is a full three-dimensional curve in SE(3) rather than a purely planar path.

To better illustrate the motion along the route, Figure 3 shows two additional views. The left plot depicts the trajectory with explicit start and end markers and sparser orientation arrows, which is useful for explaining overall motion direction. The



(a) Top-down X-Z view of the raw and Kalman-smoothed VO trajectories with camera heading arrows.

(b) 3D trajectory (X-Y-Z) comparing raw VO and Kalman-filter estimate.

Figure 2: Global trajectory estimates produced by the proposed VO pipeline.

right plot overlays stylized camera frustums along the path (projected into the X-Z plane), providing an intuitive visualization of where the virtual camera is looking as it moves through the environment.

4.2 Internal Trajectory Statistics

In addition to visual inspection, the system reports simple statistics of the translation sequence. For example, over the evaluated sequence, the raw VO produces a path of length L_{raw} (up to an unknown global scale). At the same time, the Kalman-filtered trajectory has length L_{kf} , with a modest reduction in the total path length and a reduced frame-to-frame step variance. The per-frame match counts (average, minimum, and maximum) are also logged to indicate the number of ORB correspondences supporting each relative pose estimate.

These internal diagnostics confirm that:

- A large number of inlier matches per frame supports the motion.
- The Kalman filter reduces noisy fluctuations in translation while preserving the overall shape of the trajectory.

4.3 Effect of Smoothing and Failure Modes

Qualitatively, the raw VO trajectory already follows a smooth forward-driving pattern, with a long straight segment, a curved section, and a turning maneuver. The Kalman filter primarily suppresses slight frame-to-frame jitter and short-wavelength

noise, especially in regions with fewer features or rapid rotation.

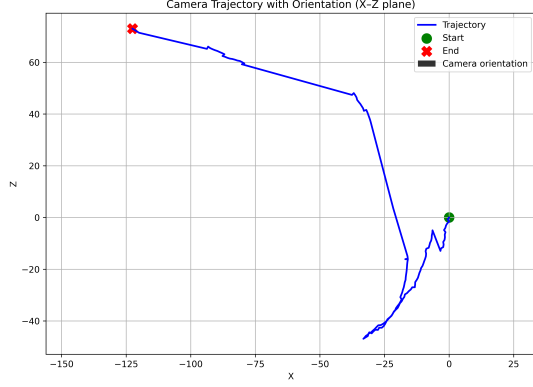
Because MonocularVO is up to an unknown global scale and there is no ground-truth trajectory for this KITTI Raw sequence, it is not possible to compute a numerical ATE or related metric. Instead, the evaluation focuses on:

- **Consistency of the global shape:** the estimated path aligns with the expected city driving pattern.
- **Stability under smoothing:** the Kalman-refined trajectory remains close to the raw estimate and does not introduce artificial drift or unrealistic motion.

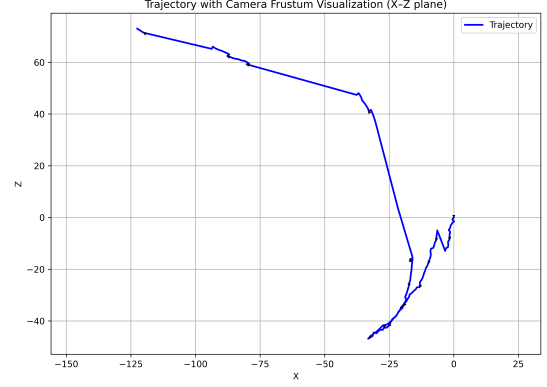
Typical failure modes for MonocularVO, such as potential scale drift over more extended sequences, degraded performance under strong motion blur, or temporary loss of tracking in low-texture regions are mitigated here by the relatively short trajectory and the use of robust feature matching and PnP-based pose refinement. However, the system could still suffer from these issues on more challenging or longer routes, which would require a stronger back-end optimization, such as full bundle adjustment or pose-graph refinement.

5 Ablation and Sensitivity Analysis

This section studies how the main design choices affect the estimated trajectory. One parameter is



(a) Trajectory with start/end markers and camera orientation arrows.



(b) Trajectory with projected camera frustums along the path.

Figure 3: Additional qualitative visualizations highlighting camera orientation along the estimated path.

Table 1: Kalman smoothing ablation (default settings otherwise).

Setting	Path length (up-to-scale)	Step mean	Step max
Kalman OFF	281.06	1.837	44.255
Kalman ON	251.56	1.655	17.088

Table 2: Effect of RANSAC inlier threshold on trajectory statistics.

Threshold (px)	Path length (up-to-scale)	Smoothed length (up-to-scale)	Rel. change (%)
$\tau = 1.0$	281.06	251.56	-10.50
$\tau = 2.0$	456.82	430.30	-5.81

varied at a time, while the others are fixed at their default values. The analysis combines trajectory statistics and qualitative inspection of the resulting paths.

5.1 Effect of Kalman Smoothing

The impact of the Kalman-based trajectory smoother is evaluated by comparing runs with the filter enabled and disabled while keeping the front-end and geometric estimation identical. The results in Table 1 show that allowing the smoother reduces both the overall path length and the maximum per-frame step size. This indicates that the smoother suppresses large, noisy jumps in the chained poses while preserving the global path structure.

Overall, Kalman smoothing shortens the path by about 10.5% and significantly reduces extreme translation spikes, consistent with its role as a temporal regularizer over the VO trajectory.

5.2 RANSAC Threshold Sensitivity

The inlier threshold τ used in the RANSAC Essential-matrix estimation strongly influences robustness. A very tight threshold ($\tau = 0.5$ px) is numerically unstable: the reported path length grows

to approximately 4.6×10^9 up-to-scale units with extremely large frame steps, clearly indicating that the model becomes dominated by a few degenerate inliers and accumulates catastrophic scale errors. This serves as a failure case where overly strict inlier selection breaks the geometry.

Table 2 compares the more reasonable settings $\tau = 1.0$ and $\tau = 2.0$ px. Increasing the threshold admits more inliers but also more outliers, leading to a longer, more drifted trajectory and less relative change after smoothing. The default value $\tau = 1.0$ provides a good trade-off between stability and plausibility of the recovered motion.

5.3 ORB Feature Count Sensitivity

The number of ORB keypoints per frame affects both the number of matches and the trajectory’s stability. Table 3 summarizes the behavior for 1000, 2000, and 3000 requested features per image. With only 1000 features, the average number of matches per frame drops to 444, and the recovered path becomes much longer with substantial maximum steps, suggesting unstable scale and drift. Increasing to 2000 features roughly doubles the average matches and yields a shorter, more coherent path. Further increasing to 3000 features again produces an overly

Table 3: Effect of ORB feature count on VO statistics.

ORB feats	Avg. matches per frame	Path length (up-to-scale)	Smoothed length (up-to-scale)	Rel. change (%)
1000	444.0	609.62	565.77	-7.19
2000	898.3	281.06	251.56	-10.50
3000	1321.0	665.94	362.16	-45.62

long trajectory and large translation jumps, likely because additional features introduce more ambiguous or dynamic points.

These experiments indicate that the chosen configuration (2000 ORB features, $\tau = 1.0$ px, Kalman smoothing enabled) offers a reasonable balance between match richness, numerical stability, and temporally coherent trajectories for the selected KITTI sequence.

6 Conclusion

This project presented a modular monocular visual odometry system evaluated on a challenging KITTI Raw driving sequence. The pipeline combines feature extraction, Essential matrix-based pose estimation with PnP refinement, SE(3) trajectory composition, and a Kalman smoothing module for temporal consistency. Ablation and sensitivity studies showed how the smoothing module, RANSAC threshold, and ORB feature count affect path length and stability, even without ground-truth poses. Overall, the results demonstrate that classical geometric vision methods, coupled with lightweight optimization and careful parameter tuning, can yield a coherent and robust 3D camera trajectory suitable as a foundation for more advanced SLAM back-ends.

References

- [1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015.
- [2] A. Geiger, J. Ziegler, and C. Stiller, “StereoScan: Dense 3D Reconstruction in Real-Time,” in *Proceedings of the IEEE Intelligent Vehicles Symposium (IV)*, 2011.